vLLM vs TensorRT-LLM: Architecture Deep-Dive

Both solve efficient LLM serving, but with fundamentally different approaches. vLLM optimizes at runtime; TensorRT-LLM optimizes at compile time.

---

vLLM and TensorRT-LLM are the two dominant frameworks for production LLM inference. Both achieve impressive throughput. Both support continuous batching. Both handle the KV cache problem. Yet they make fundamentally different architectural choices, and understanding those differences matters for choosing the right tool.

The Core Difference

vLLM: Runtime flexibility. Models load as-is, optimizations happen dynamically. The key innovation is PagedAttention, which treats KV cache like virtual memory.

TensorRT-LLM: Compile-time optimization. Models are compiled into optimized TensorRT engines with fused kernels and hardware-specific code generation. Peak performance, but upfront cost.

This isn't just an implementation detail. It shapes everything: startup time, throughput ceiling, flexibility, and operational complexity.

Memory Management

Memory management for KV cache is the defining challenge of LLM serving. Context lengths are variable, requests arrive unpredictably, and the cache dominates GPU memory.

PagedAttention (vLLM)

vLLM's key insight: treat KV cache like operating system virtual memory.

```
Traditional approach:
  Request A: [############--------]  (12 tokens, 8 wasted)
  Request B: [################----]  (16 tokens, 4 wasted)
  Request C: [####----]              (4 tokens, 4 wasted)
  → 16 slots wasted to fragmentation

PagedAttention:
  Physical blocks: [AAAA][AAAA][AAAA][BBBB][BBBB][BBBB][BBBB][CCCC]
  → Near-zero waste, non-contiguous allocation
```

Each "page" is a fixed-size block (typically 16 tokens). Requests allocate pages as needed, release them immediately on completion. The page table maps logical positions to physical memory.

Results:

- 4x more requests in same memory vs naive allocation
- Enables prefix caching (shared pages across requests with common prefixes)
- Copy-on-write for beam search and parallel sampling

TensorRT-LLM KV Cache

TensorRT-LLM adopted paged KV cache concepts (they call it "paged KV cache" or use in-flight batching with memory pools). The implementation differs (it's integrated with the compiled graph rather than managed by a Python runtime), but the core idea of non-contiguous allocation is now shared.

The difference: TensorRT-LLM's memory management is more static. Pool sizes and maximum sequence lengths are often set at compile time, trading flexibility for predictability.

Batching Strategies

Both frameworks implement continuous batching (also called "iteration-level scheduling" or "in-flight batching"). The concept:

1. Don't wait for all requests to finish
2. After each decode step, check: any requests done?
3. If yes, evict them and admit new requests
4. Process next token for remaining requests

This eliminates head-of-line blocking from variable sequence lengths.

Scheduling Differences

vLLM's scheduler runs in Python, making decisions every iteration:

- Preemption support (pause low-priority requests)
- Dynamic priority adjustment
- Flexible admission policies

TensorRT-LLM's in-flight batching is tighter:

- C++ scheduler with less per-iteration overhead
- Batching decisions can be partly compiled
- Lower flexibility, lower latency

Prefill vs Decode

LLM inference has two distinct phases:

- **Prefill**: Process the entire prompt (compute-bound, parallelizable)
- **Decode**: Generate tokens one-by-one (memory-bound, sequential)

vLLM handles these uniformly: prefill is just the first iteration with more tokens. TensorRT-LLM can compile separate optimized kernels for each phase.

Phase	vLLM	TensorRT-LLM
Prefill	Dynamic chunking	Optimized prefill kernel
Decode	Standard attention	Fused decode kernel
Mixing	Unified scheduling	Separate optimization

The Compilation Tradeoff

Here's where the philosophies diverge most sharply.

vLLM: Interpretation

```
# vLLM model loading (simplified)
model = AutoModelForCausalLM.from_pretrained("llama-70b")
engine = LLMEngine(model)
# Ready to serve in ~2 minutes (model loading)
```

vLLM loads HuggingFace models directly. Optimizations happen at runtime:

- FlashAttention kernels selected dynamically
- Batch sizes adjusted on-the-fly

- No compilation step

TensorRT-LLM: Compilation

```
# TensorRT-LLM model preparation (simplified)
# Step 1: Convert model (offline, ~30-60 minutes)
trtllm-build --model_dir llama-70b --output_dir engine_dir \
    --max_batch_size 32 --max_input_len 2048 --max_output_len 512

# Step 2: Load compiled engine (~1 minute)
engine = tensorrt_llm.runtime.ModelRunner.from_dir("engine_dir")
```

The compilation step:

- Fuses operations (e.g., QKV projection + attention + output projection)
- Generates hardware-specific CUDA kernels
- Optimizes memory layout for target GPU
- Prunes unused code paths

The cost: 30-60 minutes for a 70B model. The benefit: 15-40% higher throughput at peak.

The Usability Gap Is Closing

NVIDIA has made significant strides in reducing TensorRT-LLM's cold-start pain:

NGC Pre-built Assets: The NVIDIA NGC Catalog now offers pre-compiled engines for popular models (Llama, Mistral, etc.) on common GPU types. Download and run, no 40-minute compilation.

AutoDeploy (Beta): The biggest shift is AutoDeploy, which takes off-the-shelf PyTorch models from HuggingFace and automatically compiles them to TensorRT-LLM without manual API rewrites. It currently supports 100+ text-to-text LLMs.

```
# AutoDeploy example - feels almost like vLLM
python build_and_run_ad.py --model "meta-llama/Llama-3-8B"
```

AutoDeploy automatically handles KV-cache insertion, graph sharding for multi-GPU, and GEMM fusion. These are the tedious parts that previously required manual implementation.

Why vLLM Still Wins on Simplicity:

1. **Uniformity**: vLLM code for an A100 is identical to code for an AMD MI300X or RTX 3060. TensorRT-LLM still requires targeting specific NVIDIA chips.

2. **The Long Tail**: For niche architectures from yesterday's research paper, vLLM runs it instantly via HuggingFace integration. AutoDeploy may still hit "Unsupported Op" errors requiring manual C++ plugin work.

The Numbers

Aspect	vLLM	TensorRT-LLM
Cold start	2-5 min	30-60 min (first time)
Warm start	2-5 min	1-2 min (engine cached)
Peak throughput (A100)	~3000 tok/s	~4000 tok/s
Memory efficiency	Excellent	Very good
New model support	Hours	Days-weeks

Throughput numbers are approximate for Llama-70B with batch size 32.

Workload matters: These numbers are highly dependent on your use case:

- **Prefill-heavy** (long documents, RAG): TensorRT-LLM's specialized prefill kernels widen the gap significantly.
- **Decode-heavy** (short chatbots): The Python scheduler overhead is negligible, and the gap often shrinks to <10%.

Flexibility vs Performance

Adding New Models

vLLM: If it runs in HuggingFace Transformers, it likely works in vLLM. Custom architectures need attention mechanism compatibility, but the barrier is low.

TensorRT-LLM: New architectures require implementing the model in TensorRT-LLM's Python API, then compilation. NVIDIA provides implementations for popular models; custom architectures require engineering effort.

Quantization

Both support INT8 and INT4 quantization, but with different strengths:

vLLM is the king of community formats. Download a model from HuggingFace in GGUF, AWQ, GPTQ, or FP8, and vLLM will likely run it instantly. No conversion step, no recompilation.

TensorRT-LLM is the king of FP8 on Hopper/Blackwell. If you're running H100 or B200 GPUs, TRT-LLM's FP8 implementation is currently the gold standard for maximum throughput without accuracy loss. Quantization is part of compilation, which means more optimization opportunities (fusing quantize/dequantize), but requires rebuilding for different schemes.

Multi-GPU Inference

Both support tensor parallelism and pipeline parallelism:

vLLM: Ray-based distribution, dynamic worker management, easier multi-node setup.

TensorRT-LLM: MPI-based, tighter synchronization, compiled communication patterns. Better peak efficiency, more complex deployment.

Hardware Lock-In

This is worth highlighting:

vLLM is increasingly hardware-agnostic. It supports AMD ROCm, AWS Inferentia, and Intel Gaudi. If your cloud strategy involves switching chip providers to optimize costs, vLLM is the safer architectural bet.

TensorRT-LLM is strictly NVIDIA-only. The performance advantages come from deep integration with CUDA and TensorRT. You're trading portability for peak efficiency.

Operational Complexity

Deployment

vLLM:

```
pip install vllm
python -m vllm.entrypoints.openai.api_server --model llama-70b
# Running
```

TensorRT-LLM:

```
# Build container with TensorRT-LLM
# Download and convert model
# Compile engine for target GPU
# Deploy with Triton Inference Server (recommended)
# Running
```

vLLM is simpler. TensorRT-LLM requires more infrastructure but integrates better with NVIDIA's deployment stack (Triton, NVIDIA AI Enterprise).

Debugging

vLLM's Python runtime means standard debugging tools work. TensorRT-LLM's compiled engines are opaque: when something breaks, you're reading CUDA profiler output.

When to Use Which

Choose vLLM when:

- Rapid iteration matters (research, prototyping)
- Model architectures change frequently
- Hardware portability is required (AMD, Intel, AWS Inferentia)
- Operational simplicity is priority
- Cold start time matters

Choose TensorRT-LLM when:

- Maximum throughput on NVIDIA GPUs is critical
- Models are stable (recompilation is rare)
- Integration with NVIDIA ecosystem (Triton) is valuable
- Engineering resources for optimization are available
- Cost per token is the primary metric

Quick Reference

If you care about...	Winner	Why
Developer velocity	vLLM	Zero compilation; standard Python debugging
Cloud portability	vLLM	Runs on NVIDIA, AMD, Intel, AWS chips
Extreme scale	TensorRT-LLM	Lowest latency on H100/B200
Enterprise support	TensorRT-LLM	Part of NVIDIA AI Enterprise stack

The Convergence

The two frameworks are learning from each other:

- TensorRT-LLM adopted paged KV cache concepts
- vLLM is adding more optimized kernels
- Both now support speculative decoding
- Both support prefix caching

The philosophical difference (runtime flexibility vs compile-time optimization) remains. But the feature gap is narrowing.

For most teams, vLLM is the right starting point: lower complexity, faster iteration, good-enough performance. When you're optimizing at the margin, squeezing the last 20% of throughput from expensive GPUs, TensorRT-LLM's compilation approach pays off.

The choice isn't permanent. Many production deployments prototype with vLLM and graduate to TensorRT-LLM for high-traffic endpoints. The models are the same; only the serving layer changes.

References

1. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (SOSP 2023) - https://arxiv.org/abs/2309.06180
2. vLLM Documentation - https://docs.vllm.ai/
3. TensorRT-LLM Documentation - https://nvidia.github.io/TensorRT-LLM/
4. NVIDIA, "Automating Inference Optimizations with TensorRT LLM AutoDeploy" - https://developer.nvidia.com/blog/automating-inference-optimizations-with-nvidia-tensorrt-llm-autodeploy
5. TensorRT-LLM AutoDeploy Documentation - https://nvidia.github.io/TensorRT-LLM/torch/auto_deploy/auto-deploy.html
6. NGC Catalog: TensorRT-LLM Containers - https://catalog.ngc.nvidia.com/orgs/nvidia/teams/tensorrt-llm/containers/release
7. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models" (OSDI 2022) - https://www.usenix.org/conference/osdi22/presentation/yu